

第四章 存储器管理

1. 存储器管理功能

- 内存分配 (内存分配数据结构、内存分配功能、内存回收功能)
- 内存保护 (硬件检查越界、软件中断处理)
- 地址映射 (逻辑、物理地址, 硬件支持)
- 内存扩充 (虚拟存储技术)
- 存储器通常包括外存、内存、cache、寄存器

2. 程序的装入和链接

- 源程序从编辑到内存中执行的三个过程: 编译、链接和装入
 - 静态链接——装入前链接成一个完整装入模块
 - 链接——装入时动态链接——运行前边装入边链接
 - 运行时动态链接——运行时需要目标模块才装入并链接
- 绝对装入——编译时产生绝对地址, 即程序逻辑地址与内存物理地址完全相同
- 装入——可重定位装入——装入时将逻辑地址转换为物理地址
- 动态运行时装入——运行时将逻辑地址转换为物理地址, 需设置定位寄存器

2.2 连续分配方式

- 连续分配: 为作业 (进程) 分配地址连续的存储空间
 - 分区管理方式
 - 固定分区分配
 - 动态分区分配
 - 可重定位分区分配
- 离散分配: 为作业 (进程) 分配不连续存储空间
 - 分页 (页式) 存储管理方式
 - 分段 (段式) 存储管理方式——段页式存储管理方式
 - 虚拟存储器
 - 请求分页存储管理方式
 - 请求分段存储管理方式
- 连续分配管理
 - 外部碎片: 内存中的某些空闲分区由于太小而难以利用
 - 内部碎片: 分配给某进程的内存区域中, 有些部分没有利用上
 - 单一连续分配
 - 只支持单道程序, 内存分为系统区和用户区, 用户程序放在用户区
 - 无外部碎片, 有内部碎片
 - 固定分区分配
 - 支持多道程序, 内存用户空间分为若干个固定大小的分区, 每个分区只能装一道作业
 - 无外部碎片, 有内部碎片
 - 两种分区方式
 - 分区大小相等
 - 分区大小不等
 - 动态分区分配
 - 支持多道程序, 在进程装入内存时, 根据进程的大小动态地建立分区
 - 无内部碎片, 有外部碎片
 - 回收区之后有相邻的空闲分区
 - 回收区之前有相邻的空闲分区
 - 回收区前、后都有相邻的空闲分区
 - 回收区前、后都没有相邻的空闲分区
 - 算法
 - 最佳置换算法 (OPT)
 - 先进先出 (FIFO) 置换算法
 - 最近最久未使用算法 (LRU)
 - CLOCK 置换算法 (NRU)
 - 改进型 CLOCK (改进型 NRU)
 - 动态重定位分区分配——通过“紧凑”技术解决动态分区分配的外部碎片问题
- 对换 (进程对换)
 - 对换空间的管理
 - 进程的换出
 - 进程的换入

3. 基本分页存储管理方式

- 页面和物理块
 - 将逻辑地址空间 (进程) 分成若干大小相等的片, 称为 页面 或 页
 - 内存空间分成与页大小相同的若干存储块, 称为块或页框
- 逻辑地址结构
 - 页号 = 逻辑地址 / 页面大小; 页内偏移量 = 逻辑地址 % 页面大小
- 页表
 - 页表记录了页面和实际存放的内存块之间的映射关系
 - 一个进程对应一张页表, 进程的每一页对应一个页表项, 每个页表项由“页号”和“块号”组成
 - 每个页表项的大小是相同的, 页表是“隐含”的
 - 得页表项存放地址 = 页表地址 + i * 页表项大小
- 地址变换结构
 - 基本地址变换结构
 - 页表寄存器的作用
 - 存放页表起始地址
 - 存放页表长度
 - 地址变换过程
 - 根据逻辑地址算出页号、页内偏移量
 - 页号的合法性检查 (与页表长度对比)
 - 若页号合法, 再根据页表起始地址、页号找到对应页表项
 - 根据页表项中记录的内存块号、页内偏移量, 得到最终的物理地址
 - 访问物理内存对应的内存单元
 - 小结
 - 页式管理中地址是一维的
 - 每存取一个数据, 要访问2次内存
 - 具有快表的地址变换结构
 - 在mmu中增设一个具有并行查询能力的特殊高速缓冲寄存器, 称 快表
- 二级和多级页表
 - 单级页表存在的问题
 - 所有页表必须连续存放, 页表过大时需要很大的连续空间
 - 在一段时间内并非所有的页面都得到, 因此没必要让整个页表常驻内存
 - 二级页表
 - 将长长的页表再分页
 - 逻辑地址结构: (一级页号, 二级页号, 页内偏移量)
 - 如何实现地址变换
 - 按照地址结构将逻辑地址拆分成三部分
 - 从PCB中读出项目录表地址, 根据一级页号查项目录表, 找到下一级页表在内存中的存放位置
 - 根据二级页号查表, 找到最终当局的内存块号
 - 结合页内偏移量得到物理地址
 - 几个细节
 - 多级页表中, 各级页表的大小不能超过一个页面, 若两级页表不够, 可以分更多级
 - 多级页表的访问次数 (假设没有快表机构)——N级页表访问一个逻辑地址需要N+1次访问

9. 请求分段存储管理方式

- 在段表基础上增加若干项
 - 存取方式: 用于标识本分段的存取属性是执行、制度、读/写
 - 访问字段: 用于记录该分段被访问的频繁程度
 - 修改位: 用于表示该段进入内存后是否被修改, 供分段置换时参考
 - 存在位: 指示本段是否已调入内存, 供程序访问时参考
 - 增补位: 用于表示该段进入内存后是否做过动态增长
 - 外存始址: 用于表示本段在外存中的起始地址, 即起始虚号
- 在请求分段系统中, 每当发现运行进程所要访问的段不在内存时, 便由缺段中断机构产生一缺段中断信号, 进入OS后由缺段中断处理程序将所需的段调入内存
- 同样需要在一条指令的执行期间, 产生和处理中断
- 缺段中断的处理比缺页中断复杂 (段不是定长)
- 一条指令执行期间可能产生多次缺段中断
- 地址变换机构
 - 在段表基础上增加若干项
 - 存取控制字段
 - 共享段表
 - 共享进程计数count
 - 段号
 - 分段的共享和保护
 - 共享段的分配与回收
 - 越界检查
 - 存取控制检查
 - 环境保护机构

8. 抖动和工作集

- 抖动现象
 - 页面频繁调入换出的现象, 主要原因是分配给进程的物理块不够
- 工作集
 - 在某段时间间隔里, 进程实际访问页面的集合
 - 驻留集大小一般不能小于工作集大小
- 驻留集
 - 指请求分页存储管理中给进程分配的内存块的集合
- 抖动的预防方法
 - 采取局部置换策略
 - 把工作集算法融入到处理机调度中
 - 利用L = S * 准则调节缺页率
 - 选择暂停的进程

7. 页面管理算法

- 最佳置换算法 (OPT)
 - 淘汰率最小, 性能最好, 但无法实现
 - 优先淘汰最长时间内不会被访问的页面
- 先进先出 (FIFO) 置换算法
 - 实现简单; 但性能很差, 可能出现Belady异常
 - 优先淘汰最先进入内存的页面
- 最近最久未使用算法 (LRU)
 - 性能很好; 但需要硬件支持, 算法开销大
 - 优先淘汰最近最久没访问的页面
- CLOCK 置换算法 (NRU)
 - 实现简单, 算法开销小; 但未考虑页面是否被修改过
 - 循环扫描页面
 - 第一轮淘汰访问位 = 0 的, 并将扫描过的页面访问位改为1, 若第一轮没选中, 则进行第二轮扫描
- 改进型CLOCK (改进型NRU)
 - 若用 (访问位, 修改位) 的形式表述, 则
 - 第一轮: 淘汰 (0, 0)
 - 第二轮: 淘汰 (0, 1), 并将扫描过的页面访问位都置为0
 - 第三轮: 淘汰 (0, 0)
 - 第四轮: 淘汰 (0, 1)

6. 请求分页存储管理方式

- 在基本分页的基础上增加了几个表项
 - 状态位: 表示页面是否已在内存中
 - 访问字段: 记录最近被访问过几次, 或记录上次访问的时间, 供置换算法选择换出页面时参考
 - 修改位: 表示页面调入内存后是否被修改过, 只有修改过的页面才需在重换时与回外存
- 缺页中断机构
 - 外存地址: 页面在外存中存放的位置
 - 找到页表项后检查页面是否已在内存, 若没在内存, 产生缺页中断
 - 缺页中断处理中, 需要将目标页调入内存, 有必要时还要换出页面
 - 缺页中断属于内中断, 属于内中断中的“故障”, 即可能被系统修复的异常
 - 一条指令在执行过程中可能会产生多次缺页中断
- 找到页表项是需要检查页面是否在内存中
 - 若页面不在内存中, 需要请求调页
 - 若内存空间不够, 还需换出页面
 - 页面调入内存后, 需要修改相应页表项
- 地址变换机构 (重点关注与基本分页不同的地方)

5. 虚拟存储器的基本概念

- 传统存储管理方式的特征、缺点
 - 一次性: 作业数据必须一次全部调入内存
 - 驻留性: 作业数据在整个运行期间都会常驻内存
- 局部性原理
 - 时间局部性: 现在访问的指令、数据在不久后很可能被再次访问
 - 空间局部性: 现在访问的内存单元周围的内存空间, 很可能在不久后会被访问
 - 高速缓存技术: 使用频繁的数据可以放到更高速的存储器中
- 虚拟内存的定义和特征
 - 程序不需全部转给即可运行, 运行时需要动态调入数据, 若内存不够, 还需换出一些数据
 - 多次性: 无需在作业运行时一次性全部装入内存, 而是允许被分成多次调入内存
 - 对换性: 无需在作业运行时一直常驻内存, 而是允许在作业运行过程中, 将作业调入、换出
 - 虚拟性: 从逻辑扩充了内存的容量, 使用户看到的内存容量, 远大于实际的容量
- 如何实现虚拟内存技术
 - 访问的信息不在内存时, 由操作系统负责将所需信息从外存调入内存 (请求调页功能)
 - 内存空间不够时, 将内存中暂时用不到的信息换出到外存 (页面置换功能)
 - 请求分页存储管理
 - 请求分段存储管理
 - 请求段页式存储管理

4. 基本分段存储管理方式

- 分段
 - 将地址空间按照自身的逻辑关系划分为若干个段, 每段从0开始编址
 - 每个段再内存中占据连续空间, 但各段之间可以不相邻
- 段表
 - 逻辑地址结构: (段号, 段内地址)
 - 记录逻辑段到实际存储地址的映射关系
 - 每个段对应一个段表项, 各段表项长度相同, 由段号 (隐含)、段长、基址组成
- 地址变换
 - 1. 由逻辑地址得到段号、段内地址
 - 2. 段号与段表寄存器中的段长度相比较, 检查是否越界
 - 3. 由段表地址、段号找到对应表项
 - 4. 根据段表中记录的段长, 检查段内地址是否越界
 - 5. 由段表中的“基址+段内地址”得到最终的物理地址
 - 6. 访问内存单元
- 分段 VS 分页
 - 页是信息的物理单位, 段则是信息的逻辑单位
 - 页的大小固定, 而段的大小不固定
 - 分页的地址空间是一维的, 分段的地址空间是二维的
 - 分页对用户不可见, 分段对用户可见
 - 分段更容易实现信息的共享和保护
- 分段 + 分页
 - 将地址空间按照程序自身的逻辑关系划分为若干个段, 再将各段分为大小相等的页面
 - 将内存空间分为与页面大小相等的一个个内存块, 系统以块为单位进行内存分配
 - 逻辑地址结构: (段号, 页号, 页内偏移量)
- 段表、页表
 - 每个段对应一个段表项, 各段表项长度相同, 由段号 (隐含)、页表长度、页表存放地址 组成
 - 每个页表对应一个页表项, 各页表项长度相同, 由页号 (隐含)、页面存放的内存块号 组成
- 段页式存储管理方式
 - 地址变换
 - 1. 由逻辑地址得到段号、页号、页内偏移量
 - 2. 段号与段表寄存器中的段长度相比较, 检查是否越界
 - 3. 由段表地址、段号找到对应段表项
 - 4. 根据段表中记录的页表长度, 检查页号是否越界
 - 5. 由段表中的页表地址、页号得到查询页表, 找到相应的页表项
 - 6. 由页面存放的内存块号、页内偏移量得到最终的物理地址
 - 7. 访问目标单元
 - 访问一个逻辑地址所需访问次数
 - 第一次——查段表, 第二次——查页表, 第三次——访问目标单元
 - 可引入快表机构, 以段号和页号为关键字查询快表, 即可直接找到最终的目标页面存放位置, 引入快表后仅需一次访问